Software Entwurf 4
Übung 1

HSD FH-HAGENBERG

Sommersemester 2026

Name:	Simon Offenberger	Abgabetermin:	KW 13
Mat.Nr.:	S2410306027	Punkte:	
Übungsgruppe:	1	korrigiert:	
Aufwand in Mh:	8		

Beispiel 1 (2+2 Punkte) Verständnisfrage: In einem Multithreaded-Programm haben Sie eine Funktion, wo (a) eine lokale Variable und alternativ dazu (b) eine statische lokale Variable deklariert ist. Erläutern Sie, ob es hier zu Problemen kommen kann - wenn ja, warum, wenn nein, warum nicht.

Beispiel 2 (4+1+2 Punkte) Main-Thread und Worker-Thread: (a) Schreiben Sie ein Multithreaded-Programm in C++, das neben dem Main-Thread weitere Worker-Threads hat. Die Worker-Threads rechnen jeweils die Fibonacci-Zahlen im Bereich 0 bis n und geben das Ergebnis auf der Konsole aus. Der Main-Thread übernimmt nur Organisatorisches (Threaderzeugen, Grenzwert n vorgeben, etc.). Die Anzahl der Worker-Threads setzen Sie dynamisch zur Laufzeit auf den Wert der vorhandenen logischen Kerne ihres Prozessors. Dazu verwenden Sie `std::thread::hardware_concurrency()` aus der C++-Thread-Library (`#include <thread>`). Bei Werten unter 4 setzen Sie die Threadanzahl zumindest auf 4.

Die Ausgabe besteht aus der ID des jeweiligen Threads und einem Text mit dem aktuellen Ergebnis:

```
WorkerThread 213 created successfully!
WorkerThread 257 created successfully!
WorkerThread 284 created successfully!
WorkerThread 292 created successfully!
Thread-ID 257: Result of fib(0) = 0
Thread-ID 213: Result of fib(0) = 0
Thread-ID 213: Result of fib(1) = 1
Thread-ID 257: Result of fib(1) = 1
Thread-ID 284: Result of fib(0) = 0
Thread-ID 292: Result of fib(0) = 0
Thread-ID 257: Result of fib(2) = 1
...
```

(b) Platzieren Sie zwischen der Ausgabe der Thread-ID, dem Fib-Ergebnis und an anderen Stellen die Anweisung `Sleep(0)` um Zeitverzögerungen zu simulieren. Testen Sie das Programm nun mehrmals und beobachten Sie die Ausgabe. Was können Sie zur Reihenfolge der Ausgaben und zur Reproduzierbarkeit sagen?

(c) Beschleunigung für ihren Chip mit k logischen Kernen: Rechnen Sie zuerst sequenziell k-Mal Fib(n) hintereinander in einem einzelnen Thread. Danach parallel Fib(n) mit k Workerthreads. Geben Sie die beiden Rechenzeiten aus und ermitteln Sie den Speedup.

Bsp für Dualcore (k=2): *Sequenziell:* Fib(n) Fib(n) *Parallel:* *Thread 1:* Fib(n) *Thread 2:* Fib(n)

Beispiel 3 (10+3 Punkte) Multithreaded Matrixmultiplikation: ***(a)*** Implementieren Sie eine multithreaded Matrixmultiplikation für eine quadratische Matrix der Dimension n in der Programmiersprache C++. Die Dimension ist eine beliebige aber fixe Konstante (Compilezeit). Die Matrizen enthalten ganze Zahlen. Verwenden Sie für jedes Element der Ergebnismatrix einen eigenen Thread. Überlegen Sie sich dabei, ob die Threads parallel (dh. „gleichzeitig") in die Ergebnismatrix schreiben dürfen.

Nachfolgend wird das Prinzip der Multiplikation anhand einer 3x3-Matrix gezeigt:

$$\begin{pmatrix} a11 & a12 & a13 \\ a21 & a22 & a23 \\ a31 & a32 & a33 \end{pmatrix} \begin{pmatrix} b11 & b12 & b13 \\ b21 & b22 & b23 \\ b31 & b32 & b33 \end{pmatrix} =$$

$$\begin{pmatrix} a11.b11 + a12.b21 + a13.b31 & a11.b12 + a12.b22 + a13.b32 & a11.b13 + a12.b23 + a13.b33 \\ a21.b11 + a22.b21 + a23.b31 & a21.b12 + a22.b22 + a23.b32 & a21.b13 + a22.b23 + a23.b33 \\ a31.b11 + a32.b21 + a33.b31 & a31.b12 + a32.b22 + a33.b32 & a31.b13 + a32.b23 + a33.b33 \end{pmatrix}$$

Bsp:

$$\begin{pmatrix} 1 & -2 & 3 \\ 4 & -5 & -6 \\ -7 & 8 & 9 \end{pmatrix} x \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix} = \begin{pmatrix} 6 & 4 & 2 \\ -12 & -5 & 2 \\ 12 & 2 & -8 \end{pmatrix}$$

Testen Sie ihr Programm mit dem angeführten Beispiel und mit eigenen Eingaben. Geben Sie diese Tests auch ab.

(b) Beantworten Sie zudem folgende Fragen (mit Begründung):

(1) Kann es zu einer Racecondition kommen?
(2) Kann gleichzeitig zur Multiplikation ohne das Ergebnis zu verfälschen
  a) die Spur[1)] der Matrix A berechnet werden?
  b) die Matrix B transponiert[2)] werden?

Allgemeine Hinweise: Legen Sie bei der Erstellung Ihrer Übung großen Wert auf eine saubere Strukturierung und auf eine sorgfältige Ausarbeitung! Dokumentieren Sie alle Schnittstellen und versehen Sie Ihre Algorithmen an entscheidenden Stellen ausführlich mit Kommentaren! Testen Sie ihre Implementierungen ausführlich! Geben Sie den **Testoutput** mit ab! Erstellen Sie eine **Funktionsbeschreibung** in LaTeX, in der Sie die Beantwortung der Fragen aufnehmen können!

[1)] Spur einer Matrix A: $Sp(A) = (a_{11}) + (a_{22}) + \ldots + (a_{nn}) = \sum_{i=1}^{n}(a_{ii})$

[2)] Eine transponierte Matrix A^T erhält man durch Austauschen von Spalten und Zeilen der Matrix A. Es gilt: $(a_{ij})^T = (a_{ji})$

1 Beispiel 1 Verständnis Frage:

In einem Multithreaded-Programm haben Sie eine Funktion, wo (a) eine lokale Variable und alternativ dazu (b) eine statische lokale Variable deklariert ist. Erläutern Sie, ob es hier zu Problemen kommen kann - wenn ja, warum, wenn nein, warum nicht.

1.1 Antwort (a) lokale Variable:

Wird in einem Multithreaded-Programm eine lokale Variable in einer Funktion deklariert, so existiert diese pro Thread separat. Jeder Thread hat seinen eigenen Stack, auf dem die lokalen Variablen gespeichert werden. Daher gibt es keine Probleme mit der Verwendung von lokalen Variablen in einem Multithreaded-Programm.

1.2 Antwort (b) statische lokale Variable:

Wird eine statische lokale Variable in einer Funktion deklariert, so existiert diese nur einmal im Speicher (im Daten-Segment des Prozesses) und wird von allen Threads gemeinsam verwendet. Dies kann zu Problemen führen, da Änderungen an der Variablen von einem Thread vorgenommen werden können, während ein anderer Thread darauf zugreift.

2 Beispiel 2 Multithreaded Fibonacci:

2.1 (a) Implementierung:

2.1.1 Fib.hpp

```cpp
/*****************************************************************//**
 * @file Fib.hpp
 * @brief Declarations for Fibonacci benchmarking and concurrent printing.
 *
 * Provides functions to benchmark recursive Fibonacci calculations
 * and to print Fibonacci sequences using multiple threads.
 ********************************************************************/
#ifndef FIB_HPP
#define FIB_HPP


/**
 * @brief Run a benchmark for computing fib(max_fib_n) in single- and multi-threaded modes.
 * @param max_fib_n Maximum Fibonacci input to compute.
 */
void BenchmarkFib(const size_t max_fib_n);

/**
 * @brief Print the Fibonacci sequence up to max_fib_n using multiple worker threads.
 * @param max_fib_n Maximum Fibonacci input to compute and print.
 */
void PrintFibSequenceMultiThreaded(const size_t max_fib_n);


#endif // !FIB_HPP
```

2.1.2 Fib.cpp

```cpp
/*****************************************************************//**
 * @file Fib.cpp
 * @brief Implementation of simple Fibonacci helpers and multi-threaded demos.
 *
 * Contains recursive fibonacci implementation and
 * benchmark/printing helpers that spawn Windows threads.
 ********************************************************************/
#include "Fib.hpp"
#include <thread>
#include <algorithm>
#include <chrono>
#include <iostream>
#include <conio.h>
#include <Windows.h>
#include <winerror.h>
#include <string>
#include "StopWatch.h"
#include "Hlp.h"


using namespace std;

const string ERROR_CREATING_THREAD = "Failed to create thread: ";

/**
 * @brief Print the computed Fibonacci value from a worker thread.
 *
 * Prints a single line identifying the worker thread and the computed
 * Fibonacci value for the given input.
 *
 * @param n Input index to fib().
 * @param fib Result of fib(n).
 */
void Print_Fib(const size_t n,const size_t fib) {
  std::cout.flush();
  std::cout << "WorkerThread " << std::this_thread::get_id();
  std::cout << ": Result of fib(" << n << ") = " << fib << std::endl;
  std::cout.flush();
}

/**
 * @brief Recursive Fibonacci implementation.
 *
 * Computes the nth Fibonacci number using a recursive algorithm.
 *
 * @param n Fibonacci index.
 * @return fib(n).
 */
size_t fib(const size_t n) {

  if (n <= 1) return n;

  return fib(n - 1) + fib(n - 2);
}

/**
 * @brief Thread procedure to calculate a single fib(n) and print the result.
 *
 * This function is intended to be used as a Windows thread entry point.
 * It expects a pointer to a `size_t` containing the input `n`. The thread
 * computes `fib(n)` and prints the result via `Print_Fib`.
 *
 * @param n Pointer to `size_t` containing the input n (passed as LPVOID).
 * @return DWORD Thread exit code (0 on success).
 */
DWORD WINAPI calculate_fib(LPVOID n) {

  const size_t num = *(static_cast<size_t*>(n));

  Print_Fib(num, fib(num));

  return NO_ERROR;
}

/**
 * @brief Thread procedure to print the full Fibonacci sequence 0..max_n.
 *
 * This function is intended to be used as a Windows thread entry point.
 * It expects a pointer to a `size_t` containing the maximum `n`. The thread
 * iterates from 0..max_n computing and printing each Fibonacci value.
 *
 * @param max_n Pointer to `size_t` containing the maximum n (passed as LPVOID).
 * @return DWORD Thread exit code (0 on success).
 */
DWORD WINAPI calculate_fib_sequence(LPVOID max_n) {

  const size_t max_num = *(static_cast<size_t*>(max_n));

  for (size_t i = 0; i <= max_num; ++i) {
    Print_Fib(i, fib(i));
  }

  return NO_ERROR;
}


/**
 * @brief Spawn multiple threads that print the Fibonacci sequence up to `max_fib_n`.
 *
 * Determines the number of logical processors and spawns one worker thread per
 * logical processor (with a minimum fallback of 4 threads). Each worker thread
 * executes `calculate_fib_sequence` with a pointer to `max_fib_n`.
 *
 * Note: This function throws `std::runtime_error` if a thread cannot be created.
```

```
 *
 * @param max_fib_n Maximum Fibonacci index to print (each thread prints 0..max_fib_n).
 */
void PrintFibSequenceMultiThreaded(const size_t max_fib_n) {

  // get the number of logical processors available on the system
  // if the number of logical processors is less than 4, use 4 threads as a fallback (for older systems)
  const size_t num_of_processors = std::thread::hardware_concurrency();
  const size_t min_threads = 4;
  const size_t num_of_threads = num_of_processors < min_threads ? min_threads : num_of_processors;
  size_t fib_n = max_fib_n;

  // vectors to store the handles and thread IDs of the worker threads
  vector<HANDLE> worker_threads;
  vector<DWORD> thread_ids(num_of_threads, 0);

  // reserve space in the vectors to avoid reallocations
  worker_threads.reserve(num_of_threads);

  cout << "Detected␣" << num_of_processors << "␣logical␣processors." << endl;

  for (size_t i = 0; i < num_of_threads; ++i) {

    // create a new worker thread and call the calculate_fib_sequence function
    // handle -> void* address of the thread
    HANDLE Worker = CreateThread(
      0,
      0, // default stack size (1MB), value determines the number of bytes
      calculate_fib_sequence,
      &fib_n,
      0,
      &thread_ids.at(i)
    );

    if (Worker == nullptr) {
      for (const HANDLE hThread : worker_threads) {
        WaitForSingleObject(hThread, INFINITE);
        if (!CloseHandle(hThread)) {
          cerr << "CloseHandle␣failed␣while␣cleanup:␣" << Hlp::ErrMsg(GetLastError());
        }
      }
      throw runtime_error(ERROR_CREATING_THREAD + Hlp::ErrMsg(GetLastError()));
    }

    worker_threads.emplace_back(Worker);

    cout << "Created␣thread␣with␣ID:␣" << thread_ids.at(i) << endl;
  }


  // Wait for all worker threads to finish and close their handles
  for (const HANDLE hThread : worker_threads) {

    const DWORD waitResult = WaitForSingleObject(hThread, INFINITE);

    if (waitResult == WAIT_FAILED) {
      cerr << "WaitForSingleObject␣failed:␣" << Hlp::ErrMsg(GetLastError());
    }

    // Return Value of the thread
    DWORD exitCode = NO_ERROR;
    if (!GetExitCodeThread(hThread, &exitCode)) {
      cerr << "GetExitCodeThread␣failed:␣" << Hlp::ErrMsg(GetLastError());
    }
    else if (exitCode != NO_ERROR) {
      cerr << "Thread␣exited␣with␣error:␣" << Hlp::ErrMsg(exitCode);
    }

    if (!CloseHandle(hThread)) {
      cerr << "CloseHandle␣failed:␣" << Hlp::ErrMsg(GetLastError());
    }
  }
}



/**
 * @brief Benchmark single-threaded vs multi-threaded Fibonacci computation.
 *
 * Measures the time to compute `fib(max_fib_n)` `k` times in both single-threaded
 * and multi-threaded modes, where `k` equals the number of logical processors
 * (with a minimum fallback of 4). The multi-threaded version spawns one thread
 * per processor that executes `calculate_fib`.
 *
 * Note: This function throws `std::runtime_error` if a thread cannot be created.
 *
 * @param max_fib_n Fibonacci index to compute in benchmarks.
 */
void BenchmarkFib(const size_t max_fib_n) {

  const size_t num_of_processors = std::thread::hardware_concurrency();
  const size_t min_threads = 4;
  const size_t num_of_threads = num_of_processors < min_threads ? min_threads : num_of_processors; // Fallback to 4 threads on older systems
  size_t fib_n = max_fib_n;

  // vectors to store the handles and thread IDs of the worker threads
  vector<HANDLE> worker_threads;
  vector<DWORD> thread_ids(num_of_threads, 0);
  // reserve space in the vectors to avoid reallocations
  worker_threads.reserve(num_of_threads);


  // c)
  // Calculation of the speedup of the parallelized version compared to a single-threaded version
  // first calculate k-Times the fib of (n) in a single-threaded version and measure the time
  // then calculate k-Times the fib of (n) in a multi-threaded version and measure the time
```

```
  // Single Threaded Version k-Times fib(n)
  cout << "Calculating fib(" << max_fib_n << ") in a single-threaded version..." << endl;

  // start the stopwatch to measure the time taken for the single-threaded version
  double single_threaded_time = 0.0;
  double multi_threaded_time = 0.0;

  stw::Start();

  for (size_t i = 0; i < num_of_threads; ++i) {
    cout << "Single Threaded: Iteration " << i << " of " << num_of_threads << " fib(" << max_fib_n << ") = " << fib(max_fib_n) << endl;
  }
  single_threaded_time = stw::Stop_ms();

  cout << endl << "Time taken for single-threaded version: " << single_threaded_time << " ms" << endl;

  // Multi Threaded Version k-Times fib(n)

  cout << "Calculating fib(" << max_fib_n << ") in a multi-threaded version..." << endl;

  stw::Start();

  for (size_t i = 0; i < num_of_threads; ++i) {

    // create a new worker thread and call the calculate_fib function
    // handle -> void* address of the thread
    HANDLE Worker = CreateThread(
      0,
      0, // default stack size (1MB), value determines the number of bytes
      calculate_fib,
      &fib_n,
      0,
      &thread_ids.at(i)
    );

    if(Worker == nullptr) {
      for (HANDLE hThread : worker_threads) {

        WaitForSingleObject(hThread, INFINITE);

        if (!CloseHandle(hThread)) {
          cerr << "CloseHandle failed while cleanup: " << Hlp::ErrMsg(GetLastError());
        }
      }
      throw runtime_error(ERROR_CREATING_THREAD + Hlp::ErrMsg(GetLastError()));
    }

    worker_threads.emplace_back(Worker);
  }

  // Wait for all worker threads to finish and close their handles
  for (HANDLE hThread : worker_threads) {
    const DWORD waitResult = WaitForSingleObject(hThread, INFINITE);
    if (waitResult == WAIT_FAILED) {
      cerr << "WaitForSingleObject failed: " << Hlp::ErrMsg(GetLastError());
    }

    // Return Value of the thread
    DWORD exitCode = NO_ERROR;
    if (!GetExitCodeThread(hThread, &exitCode)) {
      cerr << "GetExitCodeThread failed: " << Hlp::ErrMsg(GetLastError());
    }
    else if (exitCode != NO_ERROR) {
      cerr << "Thread exited with error: " << Hlp::ErrMsg(exitCode);
    }

    if (!CloseHandle(hThread)) {
      cerr << "CloseHandle failed: " << Hlp::ErrMsg(GetLastError());
    }
  }

  multi_threaded_time = stw::Stop_ms();

  cout << endl << "Time taken for multi-threaded version: " << multi_threaded_time << " ms" << endl;

  if (multi_threaded_time > 0) {
    cout << endl << "Speedup: " << single_threaded_time / multi_threaded_time << "x" << endl << endl;
  }
  else {
    cerr << "Multi-threaded time is zero, cannot calculate speedup." << endl;
  }
}
```

2.1.3 Funktionsbeschreibung

Für die parallele Berechnung der Fibonacci Reihen werden für jeden logischen Prozessor ein Thread erstellt.
Jeder dieser Threads berechnet die Fibonacci Folge bis zum n-ten Element.
N wird hier vom Aufrufer der Funktion **PrintFibSequenceMultithread** übergeben.
Die Ausgabe der Fibonacci Zahlen erfolgt auf der Konsole.
Die Berechnung der Fibonacci Zahlen erfolgt rekursiv.
Für die Threading Funktionalität wurde die Windows API verwendet.

Nach ersten Test ist aufgefallen, dass die Ausgabe keiner Reihenfolge folgt.
Da alle Threads gleichzeitig auf die Konsole zugreifen, kann es zu einer Vermischung der Ausgaben kommen.

2.2 (b) Sleep(0)

2.2.1 Quell Code

```cpp
/**
 * @brief Print the computed Fibonacci value from a worker thread.
 *
 * Prints a single line identifying the worker thread and the computed
 * Fibonacci value for the given input.
 *
 * @param n Input index to fib().
 * @param fib Result of fib(n).
 */
void Print_Fib(const size_t n,const size_t fib) {
        std::cout.flush();
        std::cout << "WorkerThread␣" << std::this_thread::get_id();
        Sleep(0);
        std::cout << ":␣Result␣of␣fib(" << n << ")␣=␣" << fib << std::endl;
        std::cout.flush();
}

/**
 * @brief Thread procedure to print the full Fibonacci sequence 0..max_n.
 *
 * This function is intended to be used as a Windows thread entry point.
 * It expects a pointer to a `size_t` containing the maximum `n`. The thread
 * iterates from 0..max_n computing and printing each Fibonacci value.
 *
 * @param max_n Pointer to `size_t` containing the maximum n (passed as LPVOID).
 * @return DWORD Thread exit code (0 on success).
 */
DWORD WINAPI calculate_fib_sequence(LPVOID max_n) {

        const size_t max_num = *(static_cast<size_t*>(max_n));

        for (size_t i = 0; i <= max_num; ++i) {
                Print_Fib(i, fib(i));
                Sleep(0);
        }

        return NO_ERROR;
}
```

2.2.2 Testausgabe:

```
Detected 20 logical processors.
Created thread with ID: 0
WorkerThread Created thread with ID: 0
9072: Result of fib(0) = 0
WorkerThread 9072: Result of fib(1) = 1
WorkerThread 9072: Result of fib(2) = 1
WorkerThread 9072: Result of fib(3) = 2
WorkerThread 9072: Result of fib(4) = 3
WorkerThread 9072: Result of fib(5) = 5
WorkerThread 9072: Result of fib(6) = 8
WorkerThread 9072: Result of fib(7) = 13
WorkerThread 9072: Result of fib(8) = 21
WorkerThread Created thread with ID: 0
9072Created thread with ID: 0
: Result of fib(9) = WorkerThread 32536: Result of fib(0) = 0
WorkerThread 32536: Result of fib(1) = 1
WorkerThread 32536: Result of fib(2) = 1
WorkerThread 32536: Result of fib(3) = 2
WorkerThread 32536: Result of fib(WorkerThread 41632: Result of fib(0) = 34
4) = 3
WorkerThread 18156: Result of fib(0) = 0
WorkerThread 18156: Result of fib(1) = 1
WorkerThread 18156: Result of fib(2) = 1
WorkerThread 18156: Result of fib(3) = 2
Created thread with ID: 0
0Created thread with ID:
WorkerThread 41632: Result of fib(1) = 1
```

```
WorkerThread 41632: Result of fib(2) = 1
WorkerThread 41632: Result of fib(3) = 2
WorkerThread 41632: Result of fib(WorkerThread 32536: Result of fib(5) = 54) = 30
WorkerThread 16204: Result of fib(WorkerThread 12488: Result of fib(0) = 0

WorkerThread 41632: Result of fib(5) = 5
WorkerThread 41632: Result of fib(6) = 8
WorkerThread 41632: Result of fib(7) = 13
WorkerThread 41632: Result of fib(8) = 21
WorkerThread 41632: Result of fib(WorkerThread 12488: Result of fib(1) = 1
WorkerThread 222489) = 34
WorkerThread 416320) = 0: Result of fib(0) =
WorkerThread 32536: Result of fib(6) =
WorkerThread 0
WorkerThread 22248: Result of fib(10) = 55
Created thread with ID: 0
WorkerThread 12488WorkerThread 32236: Result of fib(0) = 0
WorkerThread 32236: Result of fib(1Created thread with ID: 0
: Result of fib(2WorkerThread 43164: Result of fib(0) = 8
WorkerThread 32536) = 1
: Result of fib(1) = 10
WorkerThread 43164: Result of fib(7) = 13

WorkerThread 22248: Result of fib(2) = 1
WorkerThread 22248Created thread with ID: 0
WorkerThread 9072WorkerThread WorkerThread 32536: Result of fib(8) = 21
) = 1
WorkerThread 32236: Result of fib(2) = 1
WorkerThread 32236: Result of fib(3) = 2
WorkerThread 32236: Result of fib(4) = 3
WorkerThread 32236: Result of fib(5) = 5
WorkerThread 32236: Result of fib(6) = 8
WorkerThread 32236: Result of fib(7) = 13
WorkerThread 32236: Result of fib(8) = 21
WorkerThread 12488: Result of fib(3) = 2
WorkerThread 12488: Result of fib(4) = 3
47568: Result of fib(0) = 0
WorkerThread 47568: Result of fib(1) = 1
WorkerThread 47568: Result of fib(2) = 1
WorkerThread : Result of fib(1) = 1: Result of fib(10) = 55
47568: Result of fib(3) = 2
WorkerThread 47568: Result of fib(4) = 3
WorkerThread 47568: Result of fib(3) = 2
WorkerThread 22248: Result of fib(4) = 3
WorkerThread Created thread with ID: 0
WorkerThread Created thread with ID: 0
WorkerThread 32536: Result of fib(9WorkerThread 11156: Result of fib(0) = 0
WorkerThread 11156: Result of fib(1) = 1
WorkerThread 11156: Result of fib(2) = 112488: Result of fib(5) = 5: Result of fib(5) = 5
WorkerThread 47568: Result of fib(6) = 8
WorkerThread 47568: Result of fib(7) = 13WorkerThread 137832: Result of fib(0) = 0
```

```
WorkerThread 137832: Result of fib(1) = 1
Created thread with ID: 0
) = Created thread with ID: 0
WorkerThread 5516: Result of fib(0) = 0
WorkerThread 5516: Result of fib(1) = WorkerThread 1404: Result of fib(0) = 0
WorkerThread 11156: Result of fib(3) = 2
WorkerThread 11156: Result of fib(434
WorkerThread 32536: Result of fib(10) = 1
WorkerThread 5516: Result of fib(2) = 1
WorkerThread 5516Created thread with ID: 0
22248Created thread with ID: 0
16204: Result of fib(1) = WorkerThread 164868: Result of fib(0) = 0
WorkerThread 164868: Result of fib(1) = 1
1
WorkerThread 16204: Result of fib(2) = : Result of fib(5) = 5
WorkerThread 22248: Result of fib(6) =
WorkerThread 43164: Result of fib(
WorkerThread 47568: Result of fib(8) = 21
WorkerThread 47568: Result of fib(9) = 34
WorkerThread 47568: Result of fib(10) = 55
Created thread with ID: ) = 3
WorkerThread 11156: Result of fib(5) = 5
WorkerThread 11156: Result of fib(6) = 8
WorkerThread 11156: Result of fib(7) = 13
WorkerThread 111560
WorkerThread 8Created thread with ID: 0
551
WorkerThread 16204: Result of fib(3) =
WorkerThread 1404: Result of fib(1) = 1
Created thread with ID: 0
WorkerThread 39368: Result of fib(0) = 0
WorkerThread 39368: Result of fib(1) =
164868: Result of fib(2) = 1
2) = 1
WorkerThread 431642
WorkerThread 16204: Result of fib(4) = 3
: Result of fib(8) = 21
WorkerThread 164100: Result of fib(0) = 0
WorkerThread 164100: Result of fib(1) = WorkerThread 16204: Result of fib (5Created thread with ID: 0

WorkerThread 22248: Result of fib(7) = 13
WorkerThread 22248: Result of fib(8) = 21
WorkerThread 22248: Result of fib(9) = : Result of fib(3) = 2
WorkerThread 43164: Result of fib(4) = 3
WorkerThread : Result of fib(3) = 2
WorkerThread 1404: Result of fib(2) = 1
WorkerThread 1404: Result of fib(3) = 2
WorkerThread 1404: Result of fib(
WorkerThread 12488: Result of fib(6) = 8
WorkerThread 12488: Result of fib(7) = 13
WorkerThread 12488: Result of fib(8) = 21
WorkerThread 12488: Result of fib(9) = 34
) = 5
```

```
WorkerThread 16204: Result of fib(6) = 8
WorkerThread 16204: Result of fib(7) = 13
WorkerThread 16204: Result of fib(8) = 21
WorkerThread WorkerThread 32236: Result of fib(9) = 34
WorkerThread 322361
WorkerThread 17952: Result of fib(0) = 0
WorkerThread 17952: Result of fib(WorkerThread 137832: Result of fib(2) = 1
WorkerThread 137832: Result of fib(WorkerThread 12488: Result of fib(10) = 55
1) = 1Created thread with ID: 17952
WorkerThread 393681
WorkerThread 164100: Result of fib(2WorkerThread 164868: Result of fib(3) = 2
WorkerThread 11156: Result of fib(9) = WorkerThread 63236: Result of fib(0) = 0
WorkerThread : Result of fib(10) = 55
34
WorkerThread 11156: Result of fib(10) = 55
63236: Result of fib(1) = 1
WorkerThread 16204: Result of fib(9) = 34
WorkerThread 16204: Result of fib(10) = 55
) = 1
WorkerThread 164100: Result of fib(3) = 2
WorkerThread 16410040764: Result of fib(WorkerThread 1815643164: Result of fib(5) = 5
3) = 2
WorkerThread 137832: Result of fib(4) = 3
WorkerThread 1378320) = 0
WorkerThread 40764: Result of fib(1) = 1
WorkerThread 40764: Result of fib(2) = 1
WorkerThread 40764: Result of fib(3) = 2
4) = 3
WorkerThread 1404WorkerThread 5516: Result of fib(4) = 3
WorkerThread 5516: Result of fib(5) = 5
WorkerThread 5516: Result of fib(6WorkerThread 40764: Result of fib(4) = : Result of fib(4) = 3
WorkerThread 18156: Result of fib(5) =
WorkerThread 17952: Result of fib(2: Result of fib(2) = 1
34
WorkerThread ) = 8
WorkerThread 5516: Result of fib(7) = 13
WorkerThread 63236: Result of fib(2) = 1
WorkerThread 63236: Result of fib(3) = 2
WorkerThread WorkerThread 43164: Result of fib(6) = 8
WorkerThread 43164: Result of fib(7) = 13
WorkerThread 43164: Result of fib(: Result of fib(5) = 5
WorkerThread 137832: Result of fib(6) = 8
WorkerThread 137832: Result of fib(7) = 13
WorkerThread 137832: Result of fib(8) = 21
WorkerThread 137832: Result of fib(632363
WorkerThread 40764: Result of fib(5) = 5WorkerThread 164868: Result of fib(4) = 3
8) = 21
```

```
WorkerThread 43164: Result of fib(9) = : Result of fib(5) = : Result of fib(4) = 3
WorkerThread 164100: Result of fib(5) = 5
WorkerThread 164100: Result of fib(6) = 8
WorkerThread 164100: Result of fib(7) = 13
WorkerThread
WorkerThread 40764: Result of fib(6) = ) = 1
WorkerThread 17952: Result of fib(3) = 2
348
WorkerThread 40764: Result of fib(7) = 13
WorkerThread : Result of fib(4) = 3
WorkerThread 63236: Result of fib(5) = 5
WorkerThread 63236: Result of fib(6) = 8
WorkerThread 6323640764: Result of fib(8) = 22248: Result of fib(10) = WorkerThread 5516: Result of fib(8) = 21
WorkerThread 551621
WorkerThread 40764: Result of fib(9) = 34
WorkerThread 40764: Result of fib(10) = 55
: Result of fib(7) = 5
WorkerThread 1404: Result of fib(6) = 8
WorkerThread 55
13

WorkerThread 43164: Result of fib(10) = 55
WorkerThread 63236: Result of fib(8) = 164100: Result of fib(8) = WorkerThread 39368: Result of fib(: Result of fib(9) = 34
WorkerThread 551621
5
WorkerThread 164868: Result of fib(5) = 5
WorkerThread 164868: Result of fib(6) = 8
3) = 2
WorkerThread 39368: Result of fib(4) = 3
WorkerThread 39368: Result of fib(5) = 5
9) = 21
WorkerThread 63236: Result of fib(9) = 34
34
WorkerThread 1404: Result of fib(7) = 13
WorkerThread 1404: Result of fib(8) = 21
WorkerThread 1404: Result of fib(9) = 34
WorkerThread 1404: Result of fib(10) = 55
WorkerThread WorkerThread 164868: Result of fib(7) = 13
WorkerThread 164868: Result of fib(8) = 21
WorkerThread 164868: Result of fib(9) = 34
WorkerThread 164868: Result of fib(10) = 55
WorkerThread 39368: Result of fib(6) = 8
WorkerThread 39368: Result of fib(7) = 13
WorkerThread 39368: Result of fib(8) = 21
WorkerThread 39368: Result of fib(9) = 34
137832: Result of fib(10) = 18156: Result of fib(6) = 8
WorkerThread 55
WorkerThread 17952: Result of fib(4) = 3
WorkerThread 1795218156: Result of fib(7) = 13
WorkerThread 18156: Result of fib(8) = 21
WorkerThread 18156: Result of fib(9) = 34
WorkerThread 18156: Result of fib(10) = 55
```

```
WorkerThread WorkerThread 164100: Result of fib(9) = 34
: Result of fib(10) = 55
39368: Result of fib(5) = 5
WorkerThread 17952: Result of fib(6) = 8
WorkerThread 17952: Result of fib(7) = 13
WorkerThread 17952: Result of fib(8) = 21
WorkerThread 17952: Result of fib(9) = 34
WorkerThread 17952: Result of fib(10) = 55
WorkerThread 63236: Result of fib(10) = 55
WorkerThread 164100: Result of fib(10) = 55
: Result of fib(10) = 55
```

2.2.3 Funktionsbeschreibung:

Wie in der Testausgabe zu sehen ist, folgt die Ausgabe keiner Reihenfolge.
Da alle Threads gleichzeitig auf die Konsole zugreifen.
Wird nun Sleep(0) in die Funktion Print_Fib eingebaut, so wird der aktuelle Thread freiwillig ausgesetzt.
Dadurch wird die Ausgabe auf der Konsole noch unvorhersehbarer.
Die Reihenfolge der Ausgabe ändert sich jedes Mal, wenn das Programm ausgeführt wird. (nicht deterministisch)

2.3 (c) Speedup

2.3.1 Quell Code

```
/**
 * @brief Recursive Fibonacci implementation.
 *
 * Computes the nth Fibonacci number using a recursive algorithm.
 *
 * @param n Fibonacci index.
 * @return fib(n).
 */
size_t fib(const size_t n) {

        if (n <= 1) return n;

        return fib(n - 1) + fib(n - 2);
}

/**
 * @brief Thread procedure to calculate a single fib(n) and print the result.
 *
 * This function is intended to be used as a Windows thread entry point.
 * It expects a pointer to a `size_t` containing the input `n`. The thread
 * computes `fib(n)` and prints the result via `Print_Fib`.
 *
 * @param n Pointer to `size_t` containing the input n (passed as LPVOID).
 * @return DWORD Thread exit code (0 on success).
 */
DWORD WINAPI calculate_fib(LPVOID n) {

        const size_t num = *(static_cast<size_t*>(n));

        Print_Fib(num, fib(num));

        return NO_ERROR;
}


/**
 * @brief Benchmark single-threaded vs multi-threaded Fibonacci computation.
 *
 * Measures the time to compute `fib(max_fib_n)` `k` times in both single-threaded
 * and multi-threaded modes, where `k` equals the number of logical processors
 * (with a minimum fallback of 4). The multi-threaded version spawns one thread
 * per processor that executes `calculate_fib`.
 *
 * Note: This function throws `std::runtime_error` if a thread cannot be created.
 *
 * @param max_fib_n Fibonacci index to compute in benchmarks.
 */
void BenchmarkFib(const size_t max_fib_n) {

        const size_t num_of_processors = std::thread::hardware_concurrency();
        const size_t min_threads = 4;
        const size_t num_of_threads = num_of_processors < min_threads ? min_threads : num_of_processors; // Fallback to 4 threads on
             older systems
        size_t fib_n = max_fib_n;

        // vectors to store the handles and thread IDs of the worker threads
        vector<HANDLE> worker_threads;
        vector<DWORD> thread_ids(num_of_threads, 0);
        // reserve space in the vectors to avoid reallocations
        worker_threads.reserve(num_of_threads);


        // c)
        // Calculation of the speedup of the parallelized version compared to a single-threaded version
        // first calculate k-Times the fib of (n) in a single-threaded version and measure the time
        // then calculate k-Times the fib of (n) in a multi-threaded version and measure the time

        // Single Threaded Version k-Times fib(n)
        cout << "Calculating␣fib(" << max_fib_n << ")␣in␣a␣single-threaded␣version..." << endl;

        // start the stopwatch to measure the time taken for the single-threaded version
        double single_threaded_time = 0.0;
        double multi_threaded_time = 0.0;

        stw::Start();

        for (size_t i = 0; i < num_of_threads; ++i) {
                cout << "Single␣Threaded:␣Iteration␣" << i << "␣of␣" << num_of_threads << "␣fib(" << max_fib_n << ")␣=␣" << fib(
                     max_fib_n) << endl;
        }
        single_threaded_time = stw::Stop_ms();

        cout << endl << "Time␣taken␣for␣single-threaded␣version:␣" << single_threaded_time << "␣ms" << endl;

        // Multi Threaded Version k-Times fib(n)

        cout << "Calculating␣fib(" << max_fib_n << ")␣in␣a␣multi-threaded␣version..." << endl;

        stw::Start();

        for (size_t i = 0; i < num_of_threads; ++i) {

                // create a new worker thread and call the calculate_fib function
                // handle -> void* address of the thread
                HANDLE Worker = CreateThread(
                        0,
                        0, // default stack size (1MB), value determines the number of bytes
                        calculate_fib,
                        &fib_n,
                        0,
                        &thread_ids.at(i)
```

```
                );

                if(Worker == nullptr) {
                        for (const HANDLE hThread : worker_threads) {

                                WaitForSingleObject(hThread, INFINITE);

                                if (!CloseHandle(hThread)) {
                                        cerr << "CloseHandle failed while cleanup: " << Hlp::ErrMsg(GetLastError());
                                }
                        }
                        throw runtime_error(ERROR_CREATING_THREAD + Hlp::ErrMsg(GetLastError()));
                }

                worker_threads.emplace_back(Worker);
        }

        // Wait for all worker threads to finish and close their handles
        for (const HANDLE hThread : worker_threads) {
                const DWORD waitResult = WaitForSingleObject(hThread, INFINITE);
                if (waitResult == WAIT_FAILED) {
                        cerr << "WaitForSingleObject failed: " << Hlp::ErrMsg(GetLastError());
                }

                // Return Value of the thread
                DWORD exitCode = NO_ERROR;
                if (!GetExitCodeThread(hThread, &exitCode)) {
                        cerr << "GetExitCodeThread failed: " << Hlp::ErrMsg(GetLastError());
                }
                else if (exitCode != NO_ERROR) {
                        cerr << "Thread exited with error: " << Hlp::ErrMsg(exitCode);
                }

                if (!CloseHandle(hThread)) {
                        cerr << "CloseHandle failed: " << Hlp::ErrMsg(GetLastError());
                }
        }

        multi_threaded_time = stw::Stop_ms();

        cout << endl << "Time taken for multi-threaded version: " << multi_threaded_time << " ms" << endl;

        if (multi_threaded_time > 0) {
                cout << endl << "Speedup: " << single_threaded_time / multi_threaded_time << "x" << endl << endl;
        }
        else {
                cerr << "Multi-threaded time is zero, cannot calculate speedup." << endl;
        }
}
```

2.3.2 Testausgabe:

```
Calculating fib(43) in a single-threaded version...
Single Threaded: Iteration 0 of 20 fib(43) = 433494437
Single Threaded: Iteration 1 of 20 fib(43) = 433494437
Single Threaded: Iteration 2 of 20 fib(43) = 433494437
Single Threaded: Iteration 3 of 20 fib(43) = 433494437
Single Threaded: Iteration 4 of 20 fib(43) = 433494437
Single Threaded: Iteration 5 of 20 fib(43) = 433494437
Single Threaded: Iteration 6 of 20 fib(43) = 433494437
Single Threaded: Iteration 7 of 20 fib(43) = 433494437
Single Threaded: Iteration 8 of 20 fib(43) = 433494437
Single Threaded: Iteration 9 of 20 fib(43) = 433494437
Single Threaded: Iteration 10 of 20 fib(43) = 433494437
Single Threaded: Iteration 11 of 20 fib(43) = 433494437
Single Threaded: Iteration 12 of 20 fib(43) = 433494437
Single Threaded: Iteration 13 of 20 fib(43) = 433494437
Single Threaded: Iteration 14 of 20 fib(43) = 433494437
Single Threaded: Iteration 15 of 20 fib(43) = 433494437
Single Threaded: Iteration 16 of 20 fib(43) = 433494437
Single Threaded: Iteration 17 of 20 fib(43) = 433494437
Single Threaded: Iteration 18 of 20 fib(43) = 433494437
Single Threaded: Iteration 19 of 20 fib(43) = 433494437

Time taken for single-threaded version: 58818.5 ms
Calculating fib(43) in a multi-threaded version...
WorkerThread 216640WorkerThread 217340: Result of fib(43) = 433494437:
    ↪ Result of fib(
```

```
43) = 433494437
WorkerThread 216760: Result of fib(43) = 433494437
WorkerThread 216792: Result of fib(43) = 433494437
WorkerThread 216380: Result of fib(43) = 433494437
WorkerThread 217368: Result of fib(43) = 433494437
WorkerThread 199888: Result of fib(43) = 433494437
WorkerThread 204872: Result of fib(43) = 433494437
WorkerThread 206520: Result of fib(43) = 433494437
WorkerThread 217156: Result of fib(43) = 433494437
WorkerThread 213856: Result of fib(43) = 433494437
WorkerThread 213848: Result of fib(43) = 433494437
WorkerThread 217324: Result of fib(43) = 433494437
WorkerThread 217352: Result of fib(43) = 433494437
WorkerThread 217348: Result of fib(43) = 433494437
WorkerThread 217152: Result of fib(43) = 433494437
WorkerThread 215420: Result of fib(43) = 433494437
WorkerThread 216696: Result of fib(43) = 433494437
WorkerThread 217224: Result of fib(43) = 433494437
WorkerThread 18104: Result of fib(43) = 433494437

Time taken for multi-threaded version: 6641.53 ms

Speedup: 8.85615x
```

2.4 Funktionsbeschreibung:

In der Funktion **BenchmarkFib** wird der Speedup der parallelen Berechnung der Fibonacci Zahlen ermittelt.
Hier wird zunächst für k mal Fib(n) berechnet und die Zeit gemessen.
k ist hier die Anzahl der logischen Prozessoren.
Anschließend wird die gleiche Berechnung in einer parallelen Version durchgeführt und die Zeit gemessen Der Speedup ergibt sich durch:

Speedup = Time_Single_Threaded / Time_Multi_Threaded

In der Testausgabe ist zu sehen, dass die parallele Version deutlich schneller ist als die single-threaded Version.
Der Speedup beträgt hier ca. 8.86.
Erwartet wurde eher ein Speedup der sich nahe an k befindet.
Da die Berechnung auf einem Laptop mit effizienz und hochleistungs Kernen durchgeführt wurde, ist der Speedup niedriger als die Anzahl der logischen Prozessoren.
Außerdem bricht bei Belastung aller CPU Kerne die Taktfrequenz ein.

3 Beispiel 3: Multithreaded Matrixmultiplikation

3.1 (a) Implementierung:

3.1.1 Matrix.h

```cpp
#ifndef MATRIX_INCLUDED
#define MATRIX_INCLUDED

////////////////////////////////////////////////////////////////////////////
// @file Matrix.h
// @brief Declaration of the Matrix class which encapsulates a single matrix
//        and provides basic operations (init, multiply, compare, print).
// @author Michael Bogner
// @version Revision history:
//          - March 2026: Simon Offenberger (updates)
////
////////////////////////////////////////////////////////////////////////////

#include <fstream>
#include <string>
#include <array>

/**
 * @class Matrix
 * @brief Simple fixed-size matrix type with basic operations.
 *
 * The matrix size is controlled by the static constant `n`.
 * The class provides initialization, multiplication (multi-threaded),
 * comparison and printing utilities.
 */
class Matrix {
public:

    inline static const std::string ERROR_CREATING_THREAD = "Failed␣to␣create␣thread:␣";
    inline static const std::string ERROR_BAD_OSTREAM = "Error␣Ostream␣is␣bad!";

    // Declarations and Members
    static int const n = 3 /* 250*/ ;   // size of matrix: set this value according to your needs!

    typedef double TMatrix[n][n];        // underlying matrix storage type

    /**
     * @brief Default constructor.
     */
    Matrix() = default;

    /**
     * @brief Virtual destructor.
     */
    virtual ~Matrix() {}

    /**
     * @brief Initialize entire matrix with the given value.
     * @param val Value to write to every cell.
     */
    void Init(double const & val);

    /**
     * @brief Accessor (non-const) for the underlying matrix.
     * @return Reference to the raw matrix array.
     */
    TMatrix & GetMatrix() {
        return mMatrix;
    }

    /**
     * @brief Accessor (const) for the underlying matrix.
     * @return Const reference to the raw matrix array.
     */
    const TMatrix & GetMatrix() const {
        return mMatrix;
    }

    /**
     * @brief Matrix multiplication operator.
     * @param other Right-hand side matrix.
     * @return Result of multiplying this * other.
     *
     * The multiplication implementation uses one worker thread per result cell.
     */
    Matrix operator*(Matrix const & other) const;

    /**
     * @brief Equality operator for matrices.
     * @param other Matrix to compare with.
     * @return true if all cells are equal, false otherwise.
     */
    bool operator==(Matrix const& other) const;

    /**
     * @brief Print matrix to the given output stream in a human readable form.
     * @param ost Output stream to write to.
     * @throws std::invalid_argument if the output stream is bad.
     */
    void Print(std::ostream & ost) const;


private:
    TMatrix mMatrix = { 0 };    // the matrix
};

/**
```

```
 * @brief Stream insertion operator for Matrix.
 * @param ost Output stream.
 * @param M Matrix to print.
 * @return Reference to the stream.
 * @throws std::invalid_argument if the stream is bad.
 */
std::ostream& operator<<(std::ostream& ost, const Matrix & M);


#endif
```

3.1.2 Matrix.cpp

```cpp
////////////////////////////////////////////////////////////////////////////
// @file Matrix.cpp
// @brief Implementation of the Matrix class.
//        Provides initialization, multi-threaded multiplication (one thread per cell),
//        comparison and printing helpers.
// @author Michael Bogner
// @date   March 2026
////////////////////////////////////////////////////////////////////////////

#include <cassert>
#include <vector>
#include <algorithm>
#include "Matrix.h"
#include "windows.h"
#include "Hlp.h"
#include <iostream>

using namespace std;

// init entire matrix with value val
void Matrix::Init(double const & val) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            mMatrix[i][j] = val;
        }
    }
}


// structure to pass the parameters for the cell multiplication task to the worker thread
typedef struct {
    const Matrix* pMatrixA;
    const Matrix* pMatrixB;
    Matrix* pResultMatrix;
    size_t row;
    size_t col;
}TCellTask ;

/**
 * \brief Worker Function that calculates the result of a Matrix Muliplication
 * \brief for one Cell of the result Matrix.
 *
 * \param ptask Pointer to a TCellTask structure that contains the parameters for the cell multiplication.
 * \return
 */
DWORD WINAPI Matrix_Cell_Mul(LPVOID ptask) {

    TCellTask* task = static_cast<TCellTask*>(ptask);

    double sum = 0;
    for (size_t i = 0; i < task->pMatrixA->n; ++i) {
        sum += task->pMatrixA->GetMatrix()[task->row][i] * task->pMatrixB->GetMatrix()[i][task->col];
    }

    task->pResultMatrix->GetMatrix()[task->row][task->col] = sum;

    return 0;
}


Matrix Matrix::operator*(Matrix const & other) const
{
    // result matrix
    Matrix result;
    const size_t num_threads = n * n;

    // vectors to store the tasks, handles and thread IDs of the worker threads
    std::vector<TCellTask> tasks;
    std::vector<HANDLE> worker_threads;
    std::vector<DWORD> thread_ids(num_threads, 0);

    // reserve space in the vectors to avoid reallocations
    tasks.reserve(num_threads);
    worker_threads.reserve(num_threads);

    size_t idx = 0;

    // Start a worker thread for each cell in the result matrix
    for (size_t row = 0; row < n; ++row) {
        for (size_t col = 0; col < n; ++col) {

            // Fill the task structure with the parameters for the cell multiplication
            TCellTask task{ this, &other, &result, row, col };

            // Store the task in the vector and start a worker thread to execute the cell multiplication
            tasks.emplace_back(move(task));

            HANDLE Worker = CreateThread(
                0,
                0, // default stack size (1MB), value determines the number of bytes
                Matrix_Cell_Mul,
                &tasks.back(),
                0,
                &thread_ids.at(idx)
            );

            // check if the thread was created successfully
            if(Worker == nullptr) {
                // close handles of already created threads before throwing the exception
                for (HANDLE hThread : worker_threads) {
                    WaitForSingleObject(hThread, INFINITE);
                    CloseHandle(hThread);
                }
                throw runtime_error(ERROR_CREATING_THREAD + Hlp::ErrMsg(GetLastError()));
```

```
            }

            // Store the handle of the worker thread in the vector
            worker_threads.emplace_back(Worker);

            ++idx;
        }
    }

    // Wait for all worker threads to finish and close their handles
    for (const HANDLE hThread : worker_threads) {
        const DWORD waitResult = WaitForSingleObject(hThread, INFINITE);
        if (waitResult == WAIT_FAILED) {
            cerr << "WaitForSingleObject failed: " << Hlp::ErrMsg(GetLastError());
        }

        // Return Value of the thread
        DWORD exitCode = NO_ERROR;
        if (!GetExitCodeThread(hThread, &exitCode)) {
            cerr << "GetExitCodeThread failed: " << Hlp::ErrMsg(GetLastError());
        }
        else if (exitCode != NO_ERROR) {
            cerr << "Thread exited with error: " << Hlp::ErrMsg(exitCode);
        }

        if (!CloseHandle(hThread)) {
            cerr << "CloseHandle failed: " << Hlp::ErrMsg(GetLastError());
        }
    }

    // Return the result matrix (RVO / move)
    return result;
}

bool Matrix::operator==(Matrix const& other) const
{

    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            if (mMatrix[i][j] != other.mMatrix[i][j]) {
                return false;
            }
        }
    }

    return true;
}

// print the matrix
void Matrix::Print(std::ostream & ost) const {

    if(!ost.good()) {
        throw std::invalid_argument(ERROR_BAD_OSTREAM);
    }

    ost << std::endl;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            ost << static_cast<int>(mMatrix[i][j]) << "  ";
        }
        ost << std::endl;
    }
}


std::ostream& operator<<(std::ostream& ost, const Matrix& M)
{
    if (!ost.good()) {
        throw std::invalid_argument(Matrix::ERROR_BAD_OSTREAM);
    }

    M.Print(ost);

    return ost;
}
```

3.1.3 Funktionsbeschreibung

Für die Implementierung wurde die bereitgestellte Klasse Matrix verwendet.
Die Matrixmultiplikation wurde mittes **operator*** realisiert.
Für die Parallelisierung wird für jeder Zelle der Ergebnis Matrix ein Thread erstellt.
Dieser Thread erhält seine Daten mittels einer Struktur.
Das Ergebnis wird direkt über einen Zeiger in die Ergebnis Matrix geschrieben.
Das gleichzeitige Schreiben in die Ergebnis Matrix führt zu keinen Problemen, da jeder Thread nur auf seine eigene Zelle zugreift.
Lesen aus den Eingabe Matrizen führt ebenfalls zu keinen Problemen, da diese nicht verändert werden.

3.2 (b) Fragen

3.2.1 Kann es zu einer Racecondition kommen?

Nein!
Da jeder Thread nur auf seine eigene Zelle der Ergebnis Matrix zugreift, gibt es keine gemeinsamen Ressourcen, die von mehreren Threads gleichzeitig verändert werden könnten.
Außerdem werden die Eingabe Matrizen nicht verändert, sondern nur gelesen!.

3.2.2 Kann gleichzeitig zur Multiplikation ohne das Ergebnis zu verfälschen die Spur der Matrix A berechnet werden?

Ja!
Die Eingabe Matrizen werden durch die Multiplikation nicht verändert!
Es können beliebig viele Threads gleichzeitig auf die Eingabe Matrizen zugreifen, ohne dass es zu Problemen kommt.

3.2.3 Kann gleichzeitig zur Multiplikation ohne das Ergebnis zu verfälschen die Matrix B transponiert werden?

Nein, wenn dies Inplace geschiehen soll.
Ja, wenn die Ausgangsmatrix B nicht verändert wird, und die transponierte Matrix in einer neuen Matrix gespeichert wird.
Die Eingabe Matrizen dürfen nicht verändert werden.
Sonst wird das Ergebnis der Multiplikation verfälscht.
Es kommt zu einer Racecondition, da die Threads gleichzeitig auf die Matrix B zugreifen und mindestens ein Thread diese verändert.

3.2.4 Testausgabe:

```

*******************************************
              TESTCASE START
*******************************************

Test for Matrix Multiplication with 0 Matrix
[Test OK] Result: (Expected:
0  0  0
0  0  0
0  0  0
 == Result:
0  0  0
0  0  0
0  0  0
)

Test for Matrix Multiplication with Identity Matrix
[Test OK] Result: (Expected:
5  5  5
5  5  5
5  5  5
 == Result:
5  5  5
```

```
5  5  5
5  5  5
)

Test for Matrix Multiplication with negative Identity Matrix
[Test OK] Result: (Expected:
-5  -5  -5
-5  -5  -5
-5  -5  -5
 == Result:
-5  -5  -5
-5  -5  -5
-5  -5  -5
)

Test for Matrix Multiplication with provided Example
[Test OK] Result: (Expected:
6  4  2
-12  -5  2
12  2  -8
 == Result:
6  4  2
-12  -5  2
12  2  -8
)

Test for Matrix Multiplication with uniformly initialized matrices
[Test OK] Result: (Expected:
18  18  18
18  18  18
18  18  18
 == Result:
18  18  18
18  18  18
18  18  18
)

Test for Exception in TestCase
[Test OK] Result: (Expected: true == Result: true)

Test for Exception in output operator
[Test OK] Result: (Expected: Error Ostream is bad! == Result: Error
    ↪ Ostream is bad!)


*****************************************

TEST OK!!
```

3.2.5 Testtreiber:

```cpp
/*****************************************************************//**
 * @file main.cpp
 * @brief Test driver that runs matrix multiplication tests and
 *        Fibonacci demos/benchmarks.
 *
 * The program writes results to console and optionally to a file "Testoutput.txt".
 ********************************************************************/
#include <iostream>
#include "Matrix.h"
#include "Fib.hpp"
#include "Test.hpp"
#include <memory>
#include <cassert>
#include <sstream>
#include "vld.h"

using namespace std;

#define WriteOutputFile ON

static bool Test_Matrix_Multiplication(std::ostream & ost);

int main(void){

    ofstream output{ "Testoutput.txt" };
    if (!output.is_open()) {
        cerr << "Konnte Testoutput.txt nicht oeffnen" << TestCaseFail;
        return 1;
    }

    try {
        bool TestOK = true;

        PrintFibSequenceMultiThreaded(30);
        BenchmarkFib(33);

        TestOK = TestOK && Test_Matrix_Multiplication(cout);


        if (WriteOutputFile) {

            TestOK = TestOK && Test_Matrix_Multiplication(output);


            if (TestOK) {
                output << TestCaseOK;
            }
            else {
                output << TestCaseFail;
            }

            output.close();
        }

        if (TestOK) {
            cout << TestCaseOK;
        }
        else {
            cout << TestCaseFail;
        }
    }
    catch (const string& err) {
        cerr << err << TestCaseFail;
    }
    catch (bad_alloc const& error) {
        cerr << error.what() << TestCaseFail;
    }
    catch (const exception& err) {
        cerr << err.what() << TestCaseFail;
    }
    catch (...) {
        cerr << "Unhandelt Exception" << TestCaseFail;
    }

    if (output.is_open()) output.close();

    return 0;

}

/**
 * @brief Run tests for matrix multiplication and report results to given stream.
 * @param ost Output stream to write test details to.
 * @return true if all tests passed, false otherwise.
 *
 * The function performs:
 *  - multiplication with a zero matrix
 *  - multiplication with an identity matrix
 *  - checks for exceptions
 */
static bool Test_Matrix_Multiplication(std::ostream& ost)
{
    assert(ost.good());

    ost << TestStart;

    bool TestOK = true;
    string error_msg;

    try
    {

        unique_ptr<Matrix> A = make_unique<Matrix>();
        unique_ptr<Matrix> B = make_unique<Matrix>();
```

```
        unique_ptr<Matrix> C = make_unique<Matrix>();

        A->Init(0);
        B->Init(5);

        *C = (*A) * (*B);

        TestOK = TestOK && check_dump(ost, "Test for Matrix Multiplication with 0 Matrix", *A, *C);

        A->Init(0);
        B->Init(5);

        // create identity matrix in A
        for(size_t i= 0; i < Matrix::n; ++i) {
            A->GetMatrix()[i][i] = 1;
        }

        *C = (*A) * (*B);
        TestOK = TestOK && check_dump(ost, "Test for Matrix Multiplication with Identity Matrix", *B, *C);

        A->Init(0);
        B->Init(5);

        // create identity matrix in A
        for(size_t i= 0; i < Matrix::n; ++i) {
            A->GetMatrix()[i][i] = -1;
        }

        *C = (*A) * (*B);

        unique_ptr<Matrix> expected = make_unique<Matrix>();
        expected->Init(-5);

        TestOK = TestOK && check_dump(ost, "Test for Matrix Multiplication with negative Identity Matrix", *expected, *C);


        A->GetMatrix()[0][0] = 1;
        A->GetMatrix()[0][1] = -2;
        A->GetMatrix()[0][2] = 3;
        A->GetMatrix()[1][0] = 4;
        A->GetMatrix()[1][1] = -5;
        A->GetMatrix()[1][2] = -6;
        A->GetMatrix()[2][0] = -7;
        A->GetMatrix()[2][1] = 8;
        A->GetMatrix()[2][2] = 9;

        B->GetMatrix()[0][0] = 9;
        B->GetMatrix()[0][1] = 8;
        B->GetMatrix()[0][2] = 7;
        B->GetMatrix()[1][0] = 6;
        B->GetMatrix()[1][1] = 5;
        B->GetMatrix()[1][2] = 4;
        B->GetMatrix()[2][0] = 3;
        B->GetMatrix()[2][1] = 2;
        B->GetMatrix()[2][2] = 1;

        expected->GetMatrix()[0][0] = 6;
        expected->GetMatrix()[0][1] = 4;
        expected->GetMatrix()[0][2] = 2;
        expected->GetMatrix()[1][0] = -12;
        expected->GetMatrix()[1][1] = -5;
        expected->GetMatrix()[1][2] = 2;
        expected->GetMatrix()[2][0] = 12;
        expected->GetMatrix()[2][1] = 2;
        expected->GetMatrix()[2][2] = -8;

        *C = (*A) * (*B);

        TestOK = TestOK && check_dump(ost, "Test for Matrix Multiplication with provided Example", *expected, *C);

        A->Init(2);
        B->Init(3);

        *C = (*A) * (*B);

        // each element of C should be 2*3*Matrix::n because each row of
        // A has n elements with value 2 and each column of B has n elements with value 3
        expected->Init(2 * 3 * Matrix::n);

        TestOK = TestOK && check_dump(ost, "Test for Matrix Multiplication with uniformly initialized matrices", *expected, *C);

    }
    catch (const string& err) {
        error_msg = err;
    }
    catch (bad_alloc const& error) {
        error_msg = error.what();
    }
    catch (const exception& err) {
        error_msg = err.what();
    }
    catch (...) {
        error_msg = "Unhandelt Exception";
    }

    TestOK = TestOK && check_dump(ost, "Test for Exception in TestCase", true , error_msg.empty());
    error_msg.clear();

    try
    {
        stringstream bad_stream;
        bad_stream.setstate(ios::failbit); // make the stream bad
        unique_ptr<Matrix> A = make_unique<Matrix>();

        bad_stream << *A; // This should fail and throw an exception
    }
    catch (const string& err) {
```

```
        error_msg = err;
    }
    catch (bad_alloc const& error) {
        error_msg = error.what();
    }
    catch (const exception& err) {
        error_msg = err.what();
    }
    catch (...) {
        error_msg = "Unhandelt Exception";
    }

    TestOK = TestOK && check_dump(ost, "Test for Exception in output operator", Matrix::ERROR_BAD_OSTREAM, error_msg);
    error_msg.clear();


    ost << TestEnd;

    return TestOK;
}
```